Part3
Computer Arithmetic

Computer Organization and Assembly Language

Computer Science Department

National University of Computer and Emerging Sciences Islamabad

Decimal Arithmetic Instructions

Decimal arithmetic is performed by combining the binary arithmetic instruction with the decimal arithmetic instructions.

The decimal arithmetic instructions are used in one of the following ways:

- **To adjust the results of a previous binary arithmetic operation** to produce a valid packed or unpacked decimal result.
- **To adjust the inputs to a subsequent binary arithmetic operation** so that the operation will produce a valid packed or unpacked decimal result.

These instructions operate only on the AL or AH registers. Most utilize the AF flag.

Decimal Arithmetic Instructions

- In the **BCD system**, the weight of each digit also increases by a factor of ten—**except that a "digit" is now a four-bit binary sequence instead of the zero-through-nine digits that we're accustomed to**. In other words, we're using a binary sequence to represent decimal digits.

Decimal	Binary	BCD
132	10000100	0001 0011 0010

Decimal Arithmetic Instructions

- Packed BCD Adjustment Instructions
  - DAA (Decimal Adjust after Addition)
  - DAS (Decimal Adjust after Subtraction)
- Unpacked BCD Adjustment Instructions
  - AAA (ASCII Adjust after Addition)
  - AAS (ASCII Adjust after Subtraction)
  - AAM (ASCII Adjust after Multiplication)
  - AAD (ASCII Adjust before Division)

ASCII AND UNPACKED DECIMAL

```
Enter first number:      3402
Enter second number:  1256
The sum is:                 4658
```

We have two options when calculating and displaying the sum:

- **Convert both operands to binary**, add the binary values, and convert the sum from binary to ASCII digit strings.
- **Add the digit strings directly by successively** adding each pair of ASCII digits (2+6, 0+5,4+2, and 3+1). The sum is an ASCII digit string, so it can be directly displayed on the screen

ASCII AND UNPACKED DECIMAL

- The second option requires the use of specialized instructions that adjust the sum after adding each pair of ASCII digits.
- Four instructions that deal with ASCII addition, subtraction, multiplication, and division are as follows:

AAA	(ASCII adjust after addition)
AAS	(ASCII adjust after subtraction)
AAM	(ASCII adjust after multiplication)
AAD	(ASCII adjust before division)

AAA Instruction

- In x86 assembly language, the AAA (ASCII Adjust after Addition) instruction is used to adjust the result of adding two unpacked BCD numbers.
- This instruction is used when you want to work with BCD values in a specific range (0-9 for each nibble), making sure the addition result is adjusted correctly for BCD.

AAA Instruction

- In 32-bit Mode, the **AAA (ASCII adjust after addition)** instruction adjusts the binary result of an ADD or ADC instruction.

- Assuming that AL contains a binary value produced by adding two ASCII digits, AAA converts AL to two unpacked decimal digits and stores them in AH and AL.

- Once in unpacked format, AH and AL can easily be converted to ASCII by ORing them with 30h.

AAA Instruction

- In 32-bit Mode, the **AAA (ASCII adjust after addition)** instruction adjusts the binary result of an ADD or ADC instruction.

```
mov ah,0
mov al,'8'     ; AX = 0038h
add al,'2'     ; AX = 006Ah
aaa            ; AX = 0100h (ASCII adjust result)
or ax,3030h    ; AX = 3130h = '10' (convert to
ASCII)
```

Performing a bitwise OR with 30h (0011 0000 in binary) is a common trick used to convert a binary-encoded decimal (0–9) stored in the lower nibble of a register into its ASCII representation.

AAA Instruction

- **Adjustment:**
- If the least significant nibble (4 bits) of AL (AL lower nibble) is greater than 9 (i.e., it contains a valid unpacked decimal number), AAA adjusts the AL register and the auxiliary carry flag (AF) as follows:

If the lower nibble of AL is greater than 9 or the auxiliary carry flag (AF) is set, the AAA instruction adjusts AL by adding 6 and increments AH by 1. This ensures that AL is within the valid BCD range (0–9).

AAA Instruction

- **Adjustment:**

If lower nibble of al>9 or AF=1

- Al=al(lower nibble)+6 and higher nibble of al=0
- Ah=ah+1
- Cf=1, af=1

If lower nibble al<9

- Al=al
- Ah=ah, cf=0.af=0
- and higher nibble of al=0

AAA Instruction

38H 0011 1000
32H 0011 0010
0110 1010
6 A
AL = 6A H
Lower nibble > 9

0110 1010
0000 0110 (AL +6)
0111 0000
0000 0000 Lower nibble set to zero

Result AL: 00h
AH: 01h
AX: 0100H

Example

- 34H
- 38H
- Add al,cl
- aaa

AAS Instruction

- In 32-bit Mode, the AAS (ASCII adjust after subtraction) instruction follows a SUB or SBB

- Instruction that has subtracted one unpacked decimal value from another and stored the result in AL. It makes the result in AL consistent with ASCII digit representation.

- Adjustment is necessary only when the subtraction generates a negative result

AAS Instruction

- The AAS instruction adjusts the value in AL to make it a valid decimal value if it's not. Here's how AAS works:
  - If the lower nibble (last 4 bits) of AL is greater than 9, or if the Auxiliary Carry (AF) flag is set, AAS subtracts 6 from AL and decrements AH by 1. It then clears the upper nibble of AL.

AAS Instruction

After the SUB instruction, AX equals 00FFh. The AAS instruction converts AL to 09h and subtracts 1 from AH, setting it to FFh and setting the Carry flag.

```
.data
     val1 BYTE '8'
     val2 BYTE '9'
.code
     mov ah,0
     mov al,val1      ; AX = 0038h
     sub al,val2      ; AX = 00FFh
     aas                ; AX = FF09h
     pushf          ; save the Carry flag
     or al,30h ; AX = FF39h
     popf           ; restore the Carry flag
```

AAM Instruction

- In 32-bit Mode, the AAM (ASCII adjust after multiplication) instruction converts the binary product produced by MUL to unpacked decimal.
- The multiplication can only use unpacked decimals.
- We multiply 5 by 6 and adjust the result in AX. After adjustment, AX = 0300h, the unpacked decimal representation of 30:

AAM Instruction

1: Product which is [AL] converted to BCD format (Unpacked mul)
2: Two operands are unpacked BCD Operands
3: The Result of AAM instruction in Ax register

```
.data
     AscVal BYTE 05h,06h
.code
     mov bl,ascVal  ; first operand
     mov al,[ascVal+1] ; second operand
     mul bl                ; AX = 001Eh
     aam                    ; AX = 0300h
```

AAM Instruction

[AL] 35H
[BL] 36H

MUL BL

AND AL,OFH AL=05, Masking
AND BL,OFH BL=06 Masking
After Mul
AL = 1EH (30 in Decimal)

AAM

AL=30
[AH]=03
[AL] = 00
[AX]= 03 00

Instruction:

Mov AL, 35H
Mov BL,36H
MUL BL
AAM

AAM Instruction Steps:

[AL] → decimal
AL/10
Quotient = [AH]
Remainder = [AL]

AAM Instruction

[AL] 35H
[BL] 38H

MUL BL

AND AL,OFH AL=05, Masking
AND BL,OFH BL=08 Masking
After Mul
AL = 28H (40 in Decimal)

AAM

AL=40
[AH]=04
[AL] = 00
[AX]= 04 00

Instruction:

Mov AL, 35H
Mov BL,38H
MUL BL
AAM

AAM Instruction Steps:

[AL] → decimal
AL/10
Quotient = [AH]
Remainder = [AL]

AAD Instruction

- In 32-Bit Mode, the AAD (ASCII adjust before division) instruction converts an unpacked decimal dividend in AX to binary in preparation for executing the DIV instruction.

- We converts unpacked 0307h to binary, then divides it by 5. DIV produces a quotient of 07h in AL and a remainder of 02h in AH

AAD Instruction

Converts two unpacked BCD digits in AL, and AH registers to equivalent hexadecimal number in AL. So the decimal number 37 in hexa is 25

```
.data
     quotient  BYTE ?
     remainder BYTE ?
.code
     mov ax,0307h          ; dividend
     aad                    ; AX = 0025h
     mov bl,5              ; divisor
     div bl                ; AX = 0207h
     mov quotient,al
     mov remainder,ah
```

Questions & Answers